

stat 432 hw5

Lu Gong

HW5

All simulations use seed 111. The report shows the results and the hand-written classifier; the complete simulation code is kept in the QMD source.

question 1

a

The true value at $\mathbf{x}_0$ is:

$$f(\mathbf{x}_0) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 \approx 0.5942$$

The training data contain 400 observations. The first six are shown below.

	x1	x2	x3	y
1	0.2352207	-0.38782966	-0.2690999	-0.2458532
2	-0.3307359	-1.41787206	2.6251086	-3.3400150
3	-0.3116238	0.01509673	-0.3536630	-0.1886363
4	-2.3023457	-0.44462959	-0.7084323	-1.7967330
5	-0.1708760	-1.24522183	-0.7911034	-1.2575428
6	0.1402782	-0.03698561	0.7610504	0.1018068

The KNN predictions are calculated below.

	k	prediction
1	1	0.6078639
2	3	0.7131868
3	5	0.6845748
4	7	0.5418679
5	9	0.4921123
6	11	0.5359319
7	13	0.4923644
8	15	0.4668582
9	17	0.4775674
10	19	0.4407616
11	21	0.4444952
12	23	0.4275882
13	25	0.4579687
14	27	0.4112572
15	29	0.3840184

b

The experiment repeats 200 times, using the same training data for every k within each repetition.

Prediction matrix: 200 rows x 15 columns

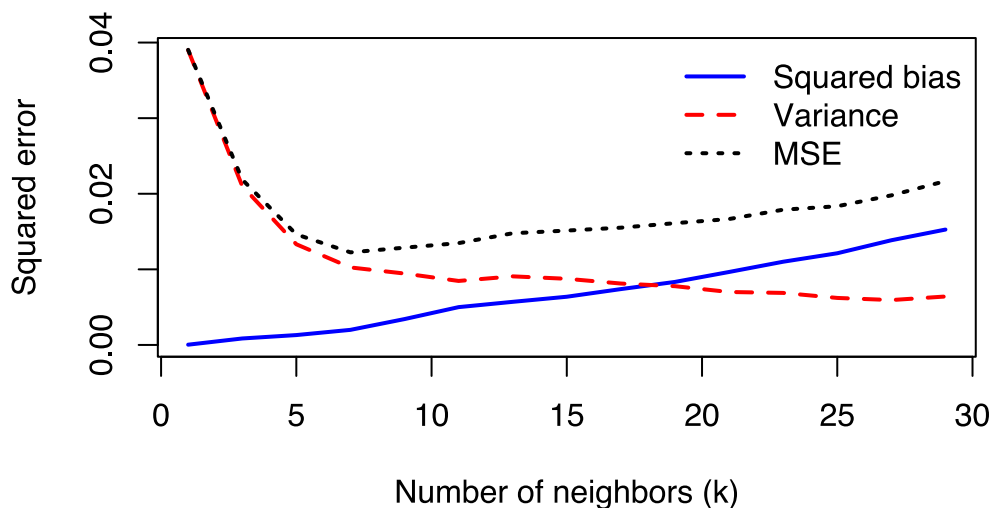
	k	bias	squared_bias	variance	MSE
1	1	-0.005711654	0.0000326230	0.039005493	0.03903812

```

2 3 -0.029251322 0.0008556398 0.021023214 0.02187885
3 5 -0.036148260 0.0013066967 0.013318136 0.01462483
4 7 -0.044670575 0.0019954603 0.010251628 0.01224709
5 9 -0.058500324 0.0034222879 0.009446064 0.01286835
6 11 -0.070730254 0.0050027688 0.008466660 0.01346943
7 13 -0.075509627 0.0057017038 0.009074583 0.01477629
8 15 -0.079861560 0.0063778687 0.008751493 0.01512936
9 17 -0.085959010 0.0073889513 0.008123750 0.01551270
10 19 -0.091305183 0.0083366365 0.007773785 0.01611042
11 21 -0.098247066 0.0096524860 0.007003720 0.01665621
12 23 -0.104919652 0.0110081333 0.006883648 0.01789178
13 25 -0.110131035 0.0121288449 0.006218425 0.01834727
14 27 -0.117684160 0.0138495614 0.005938259 0.01978782
15 29 -0.123548466 0.0152642234 0.006409546 0.02167377

```

Maximum absolute difference: $\text{MSE} - \text{squared bias} - \text{variance} = 4.163336\text{e-}17$



In this simulation, as k increases, KNN averages more neighbours, so noise in a single observation has less influence. This makes predictions more stable and lowers variance. But larger k also includes more distant neighbours, which may have different signals from the target point. So the average prediction can be further from the true value, increasing squared bias.

The MSE decreases at first because the drop in variance is larger than the increase in squared bias. At larger k , the increase in squared bias becomes more important, so MSE rises. Small fluctuations may partly come from the finite number of simulations.

KNN, ridge, and lasso can all reduce variance at the cost of more bias, but they work differently. Ridge and lasso shrink regression coefficients, reducing the fit to noise but possibly weakening the true signal too. KNN averages more neighbours by increasing k , which can also increase bias.

question 2

a

Both methods use 400 training and 1,000 test observations per setting. The lasso penalty is selected by 10-fold cross-validation over the required 41 values.

Setting	KNN_MSE	Lasso_MSE	Lambda	Nonzero
Independent	0.5727	0.2518	0.0251	10
Correlated	0.1659	0.2389	0.0126	10

b

When variables are independent, KNN uses all 30 variables to find neighbours, but only the first 3 directly affect Y. The unrelated variables can affect distances and make the selected neighbours less useful. Lasso can shrink their coefficients towards zero, reducing their influence. In this simulation, lasso has a lower MSE in the independent case.

In the correlated case, the other 27 variables carry information about the first 3, so the neighbours can be more useful for prediction. Lasso uses coefficients rather than distances and may keep some correlated variables while shrinking others to zero. In this case, KNN has a lower MSE than lasso.

c

The lasso model used here is linear in the input variables, so it cannot fully represent the sine and quadratic relationships, even with a good lambda. KNN averages responses from different neighbours at different target points, so it can adapt to nonlinear relationships. But it can still perform poorly if unrelated variables affect distances or the neighbours are not close enough.

We only generated one dataset for each setting. The results could change with a different true relationship, sample size, number of variables, or k. So this does not show that correlation always helps KNN or that one method is always better.

question 3

a

Both settings use 200 training and 200 test observations.

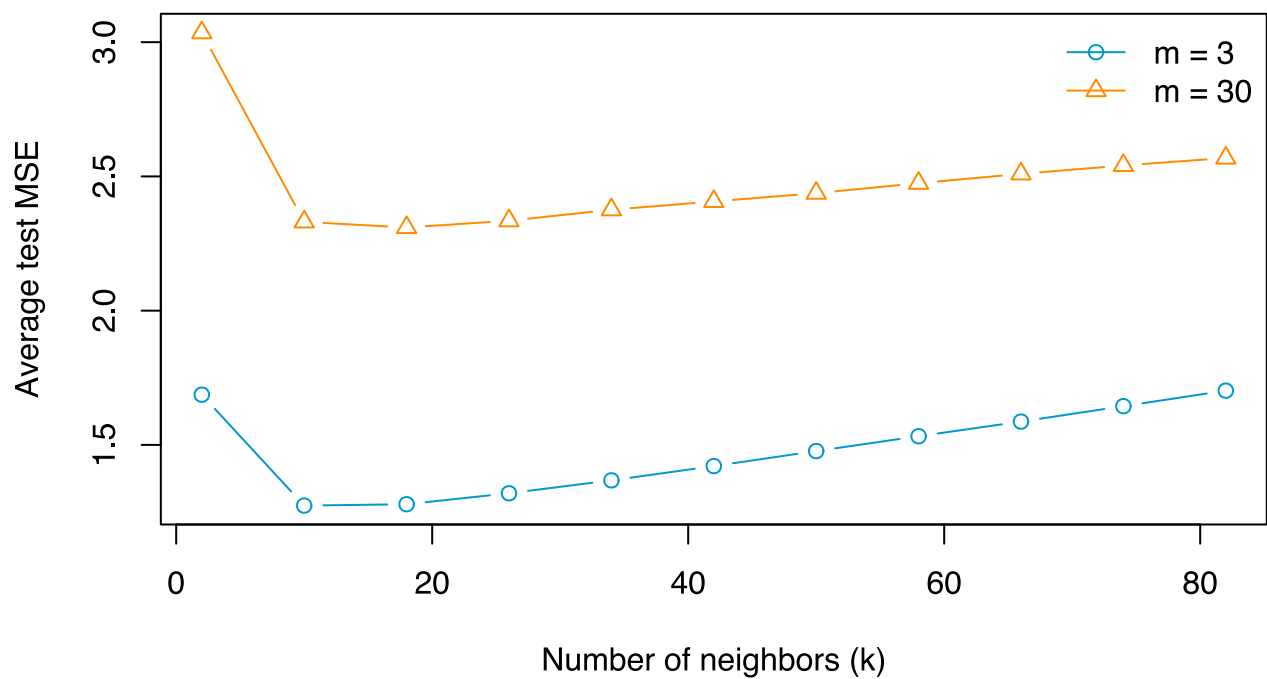
	k	m3	m30
1	2	1.727793	3.301488
2	10	1.184709	2.574498
3	18	1.178218	2.586916
4	26	1.224777	2.621227
5	34	1.282951	2.667758
6	42	1.382732	2.729464
7	50	1.449943	2.774361
8	58	1.536075	2.791732
9	66	1.620765	2.810641
10	74	1.661019	2.843727
11	82	1.706065	2.838079

b

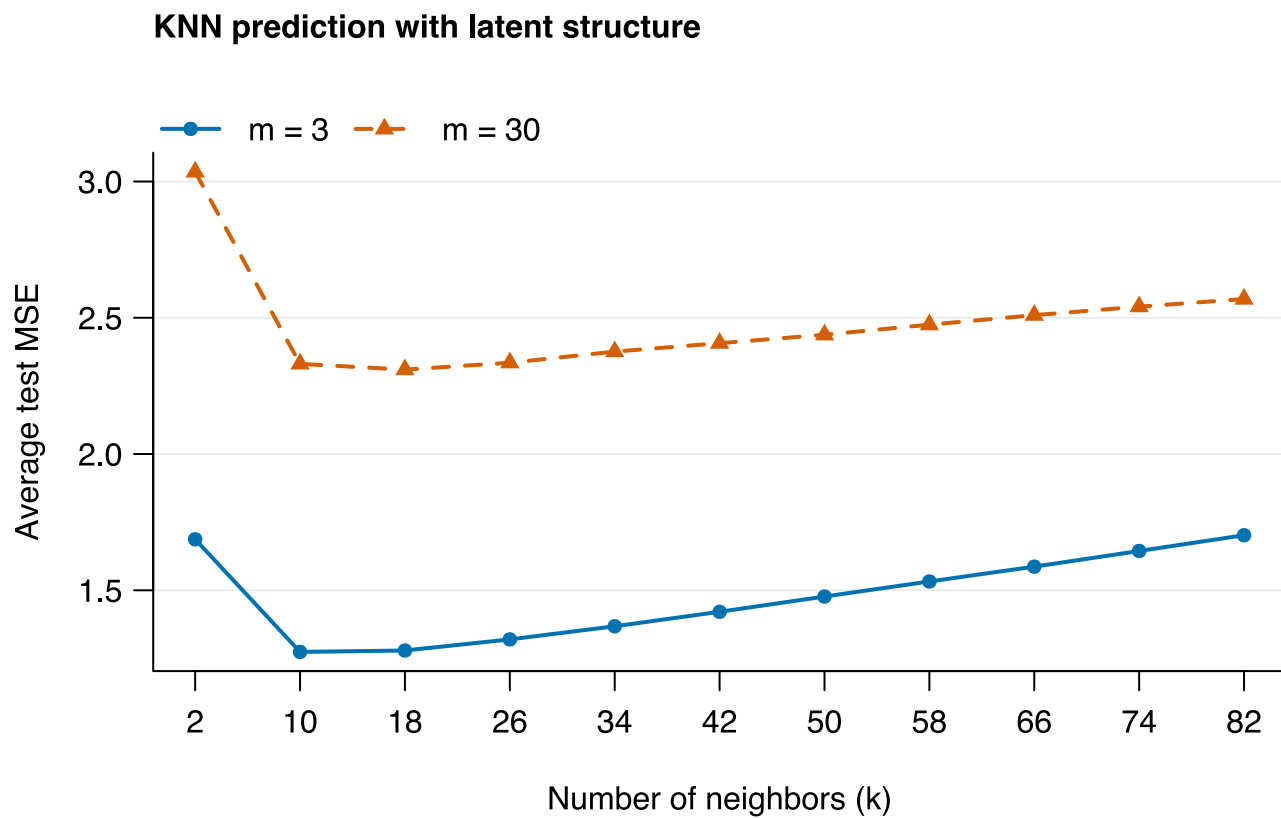
The table and curves show average test MSE over 50 repetitions.

	k	m3	m30
1	2	1.687096	3.035300
2	10	1.273943	2.330776
3	18	1.279002	2.309743
4	26	1.319823	2.334943
5	34	1.367920	2.375850
6	42	1.421055	2.406821
7	50	1.477016	2.437636
8	58	1.532292	2.474735
9	66	1.586746	2.509511
10	74	1.644234	2.540858
11	82	1.702053	2.568619

Basic formatting:



Revised formatting, using the same results and axis limits:



c

KNN uses 100 observed variables in both settings, not the latent variables directly. With $m = 30$ rather than 3, more latent directions affect distances, but only the first two directly affect Y . So the selected neighbours may be less similar in the variables that matter for prediction. Measurement noise in X can also change distances and neighbour selection.

Without measurement noise, $\mathbf{X} = \mathbf{AZ}$ is a linear combination of the m columns of A , so the data lie in a linear space with dimension at most m . Noise adds independent variation to all 100 coordinates and moves samples outside this space. So the observed data have a low-dimensional structure with noise, rather than lying exactly in that space.

question 4

a

```
archive <- "/Users/lugong/Desktop/26Fall/STAT 432/原生资料/作业原件/STAT432_HW05_官方原始材料包_2026-09-26.zip"
read_zip_data <- function(name) {
  con <- gzcon(unz(archive, name, open = "rb"), text = TRUE)
  on.exit(close(con))
  as.matrix(read.csv(con, header = FALSE))
}
zip.train <- read_zip_data("week-05/data/knn/zip-train.csv.gz")
zip.test <- read_zip_data("week-05/data/knn/zip-test.csv.gz")

X <- zip.train[1:1000, -1]
y <- zip.train[1:1000, 1]
X_test <- zip.test[1:500, -1]
y_test <- zip.test[1:500, 1]

euclidean_distance <- function(x0, X) sqrt(rowSums(sweep(X, 2, x0, "-")^2))
manhattan_distance <- function(x0, X) rowSums(abs(sweep(X, 2, x0, "-")))

knn_classify <- function(X, y, X_test, distance) {
  y_hat <- integer(nrow(X_test))
  for (i in seq_len(nrow(X_test))) {
    d <- distance(X_test[i, ], X)
    neighbors <- order(d, seq_len(nrow(X)))[1:5]
    y_hat[i] <- which.max(tabulate(y[neighbors] + 1, nbins = 10)) - 1
  }
  y_hat
}

y_hat <- cbind(Euclidean = knn_classify(X, y, X_test, euclidean_distance),
               Manhattan = knn_classify(X, y, X_test, manhattan_distance))
```

b

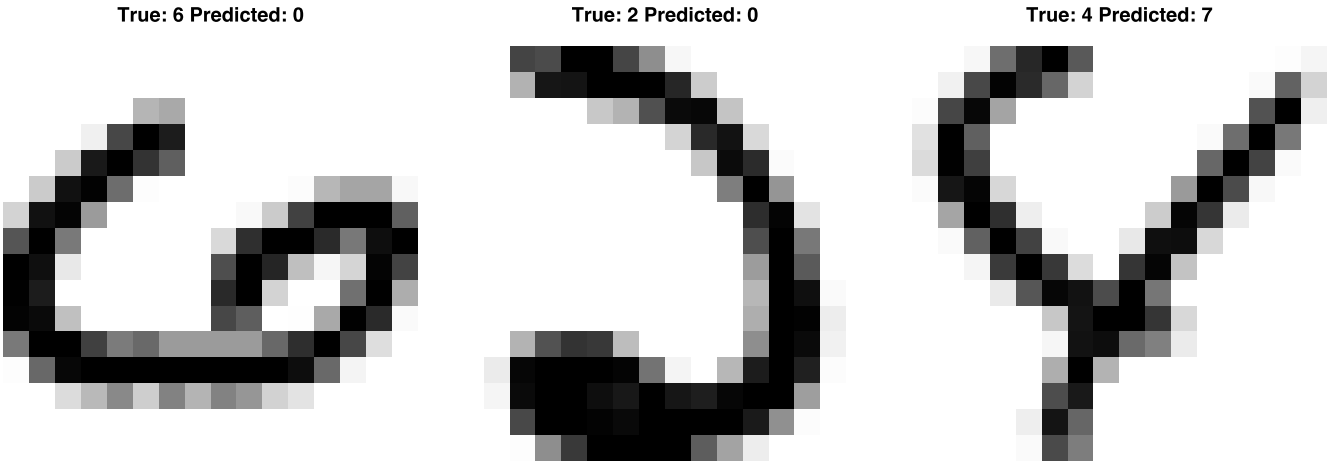
Euclidean test error: 0.122

	True									
Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0

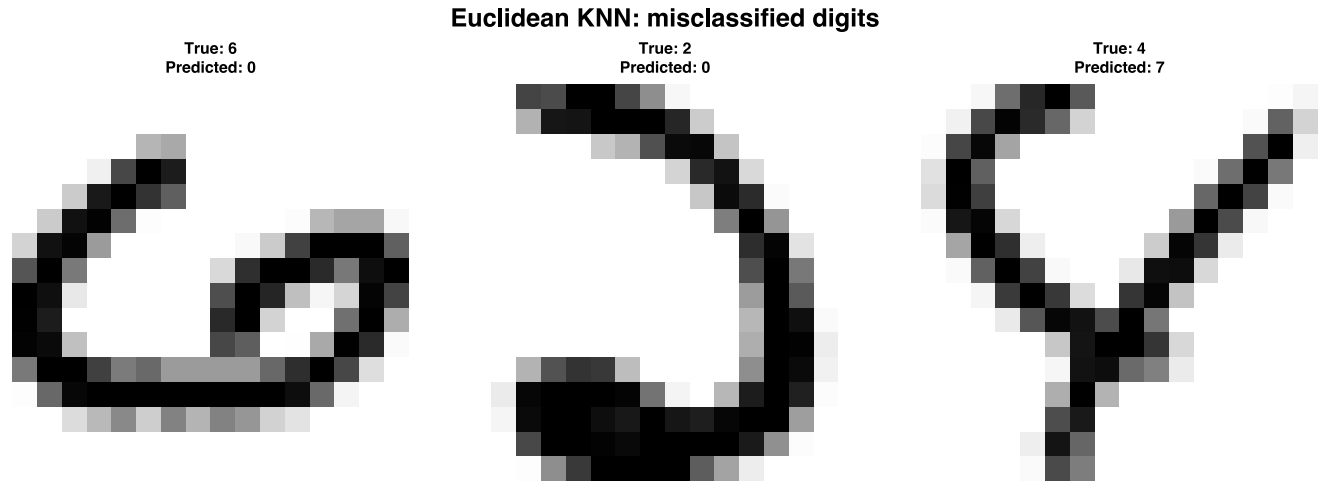
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41
Manhattan test error: 0.138										
True										
Predicted	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

Euclidean has a 12.2% error rate and Manhattan has 13.8%, so Euclidean performs better in this case. The most common mistake is predicting 2 as 0.

c
Basic formatting:



Revised formatting, using the same three images:



Different predictions: 22

	Manhattan_correct	
Euclidean_correct	FALSE	TRUE
FALSE	58	3
TRUE	11	428

The mistakes make visual sense: 2 and 6 have curves that look like 0, and the shape of 4 may look like 7. But similar error rates do not mean the methods make mistakes on the same images. Their predictions differ on 22 images: both are wrong on 8, only Euclidean is correct on 11, and only Manhattan is correct on 3. So their sets of misclassified images are different.

question 5

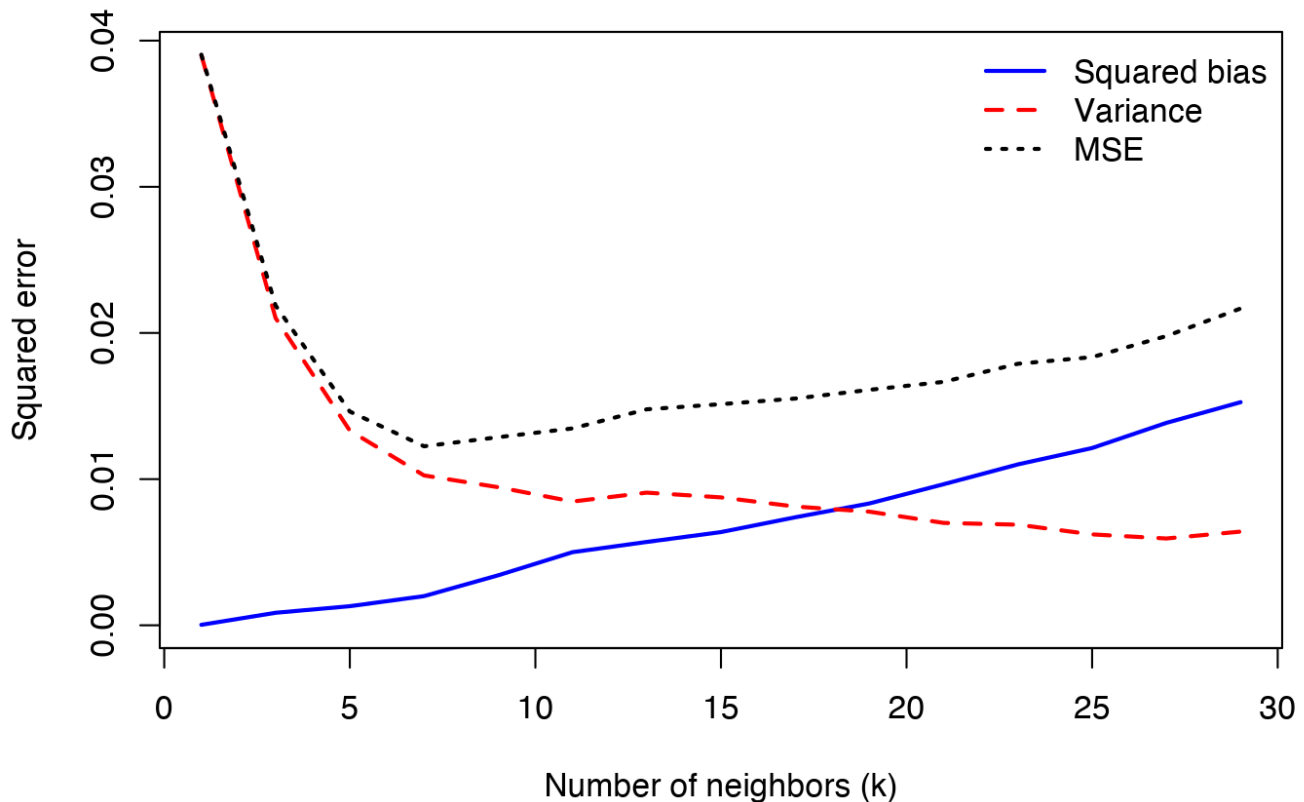
a

The skill formats STAT 432 plots in R or Python only when requested. It does not change the calculations. The final `SKILL.md` is in part (c).

b

I asked the agent to install and use the skill on the Question 1 plot. Both versions use the same results, seed 111, and axis limits. Only the margins, title, labels, colors, line widths, and legend placement change. Questions 3 and 4 also compare basic and revised formatting.

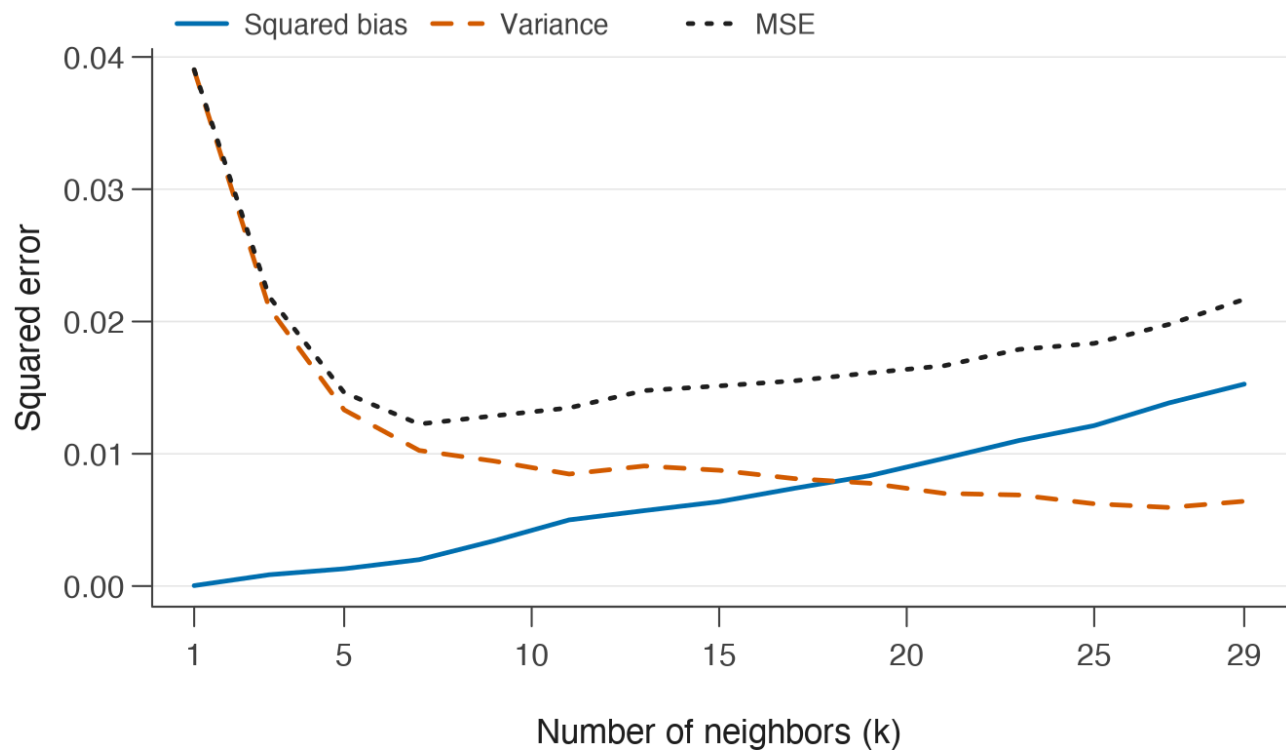
Original plot:



Revised plot:

KNN bias-variance trade-off

200 simulations | seed = 111



c

I prefer the second version because it is clearer and the fluctuations are easier to see. The data stay the same. The rules use readable labels, enough space, and distinct lines, so they also work for other STAT 432 plots. The skill and revised plots were prepared with AI assistance.

```
---
name: stat432-plot-formatting
description: Improve STAT 432 homework plots in R or Python only when explicitly requested. Do not apply
to other courses or change statistical calculations.
---

# STAT 432 Plot Formatting

- Preserve the data, random seed, model, and numerical results.
- Leave enough margins so titles, labels, and legends are not clipped.
- Use a short descriptive title and clear axis labels, including units when relevant.
- Keep axis scales appropriate and consistent between comparable plots.
- Use distinguishable, colorblind-friendly colors and different line styles.
- Choose figure dimensions and font sizes that remain readable in a homework PDF.
- Place legends where they do not cover the data.
- Compare the original and revised plots and briefly explain the formatting changes.
```